

Take-home: turn a query contract into SQL

Walt engineering exercise

CANDIDATE TAKE-HOME

The setting

At Walt, an LLM never writes SQL. It fills in a typed JSON form — we call it the **contract** — that says what the user asked: which metrics, sliced by what, filtered how, compared against what. Something then has to turn that contract into SQL that runs on the customer's warehouse. Your job is to build that something.

- Two hard constraints shape everything
- The SQL must come out in **under 10 milliseconds** on an ordinary laptop, measured from contract in to SQL string out (execution time doesn't count).
 - The same contract must always produce the **same SQL**. Byte for byte, every run.

How you get there is up to you — the shape of the solution is the exercise.

What you get

1. The schema and data

A tiny star schema. Load it into DuckDB (or any database you like) to test.

```
CREATE TABLE dim_store ( store_id TEXT PRIMARY KEY, store_name TEXT, region TEXT );
CREATE TABLE dim_calendar ( date DATE PRIMARY KEY, fiscal_year INTEGER, fiscal_quarter TEXT );
CREATE TABLE fact_sales ( -- grain: one order line
  order_id TEXT, -- an order can have several lines, in different stores
  order_date DATE, ship_date DATE, store_id TEXT,
  channel TEXT, -- 'Online' | 'Retail'
  revenue DOUBLE
);

INSERT INTO dim_store VALUES
  ('S1','Downtown','North'), ('S2','Uptown','North'),
  ('S3','Riverside','South'), ('S4','Hilltop','West');

-- fiscal year = calendar year here; populate dim_calendar for every date below
INSERT INTO dim_calendar VALUES
  ('2025-03-10',2025,'Q1'), ('2025-04-02',2025,'Q2'), ('2025-06-15',2025,'Q2'),
  ('2026-02-01',2026,'Q1'), ('2026-02-11',2026,'Q1'),
  ('2026-03-05',2026,'Q1'), ('2026-03-06',2026,'Q1'), ('2026-06-20',2026,'Q2');

INSERT INTO fact_sales VALUES
  ('0-100','2025-03-10','2025-03-12','S1','Online',100),
  ('0-101','2025-04-02','2025-04-04','S3','Retail',200),
  ('0-102','2025-06-15','2025-06-17','S4','Online',150),
  ('0-200','2026-02-01','2026-02-03','S1','Online',120),
  ('0-201','2026-02-11','2026-02-13','S2','Retail', 80),
  ('0-202','2026-03-05','2026-03-07','S3','Online',300),
  ('0-203','2026-03-06','2026-03-08','S1','Retail', 50), -- one order,
  ('0-203','2026-03-06','2026-03-08','S3','Retail', 70), -- two regions
  ('0-204','2026-06-20','2026-06-22','S4','Retail', 90);
```

2. The semantic model

Your solution never reads the warehouse and never guesses. Everything it may know about this customer's data is declared here:

```
{
  "datasets": [
    { "name": "fact_sales", "grain": "one order line" },
    { "name": "dim_store", "grain": "one store" },
    { "name": "dim_calendar", "grain": "one date" }
  ],
  "relationships": [
    { "from": "fact_sales", "from_column": "store_id", "to": "dim_store",
      "to_column": "store_id", "cardinality": "many_to_one" },
    { "from": "fact_sales", "from_column": "order_date", "to": "dim_calendar",
      "to_column": "date", "cardinality": "many_to_one" }
```

```

    ],
    "metrics": [
      { "name": "total_revenue", "agg": "sum", "expression": "revenue",
        "model": "fact_sales", "measure_class": "additive" },
      { "name": "order_count", "agg": "count_distinct", "expression": "order_id",
        "model": "fact_sales", "measure_class": "distinct_count" }
    ],
    "dimensions": [
      { "name": "region", "model": "dim_store" },
      { "name": "channel", "model": "fact_sales" },
      { "name": "fiscal_year", "model": "dim_calendar" }
    ]
  }
}

```

3. The contracts

These are lightly adapted from contracts our production system actually emits.

Contract A — aliased metrics with per-metric filters. *"Online revenue next to total revenue, fiscal 2026."*

```

{
  "metrics": [
    { "name": "total_revenue", "as": "online_rev",
      "filters": [ { "field": "channel", "op": "=", "value": "Online", "model": "fact_sales" } ] },
    { "name": "total_revenue" }
  ],
  "group_by": [],
  "filters": [
    { "field": "fiscal_year", "op": "=", "value": 2026, "model": "dim_calendar" }
  ]
}

```

Note what this asks of you

Two readings of the same metric in one query, and a filter whose field lives on a *different table* than the fact — your solution has to work out the join from the declared relationships, because nothing in the contract names it.

Expected result (one row)

online_rev = 420 , total_revenue = 710

Contract B — comparison with derived columns and a grand total. *"Revenue by region, fiscal 2026 vs 2025, with change and percent change, plus a grand total row."*

```

{
  "metrics": [ { "name": "total_revenue" } ],
  "group_by": [ "region" ],
  "compare": {
    "dimension": "fiscal_year",
    "periods": [ "2025", "2026" ],
    "primary": "2026",
    "outputs": [ "values", "delta", "pct_change" ]
  },
  "totals": "grand"
}

```

The `compare` turns the two periods into columns — the result has columns like `total_revenue_2025` , `total_revenue_2026` , `total_revenue_delta` , `total_revenue_pct_change` — one row per region, and a totals row.

Expected result

region	total_revenue_2025	total_revenue_2026	delta	pct_change
North	100	250	150	150.0%
South	200	370	170	85.0%
West	150	90	-60	-40.0%
Total	450	710	260	57.8%

Look hard at the totals row before you write any code

The value cells sum (revenue is additive). But the total's `pct_change` is **57.8%** — computed from the total row's own two value cells (260 / 450) — never the sum of the percentage column (195%) and never its average. Percent-change is row-local arithmetic; a totals row rebuilds it

from its own cells. Getting this right is part of the exercise.

Contract C. *"Distinct orders by region, with a grand total row."*

```
{
  "metrics": [ { "name": "order_count" } ],
  "group_by": [ "region" ],
  "totals": "grand"
}
```

Expected

North 4, South 3, West 2 — and the totals row says **8, not 9**. Order O-203 has lines in two regions, so it appears in two groups; adding the groups double-counts it. A distinct count's total must be recounted against the base at the total's own grain, never assembled from the rows above it.

If you run short on time

You may cover this one in the README instead of in code — explain how your solution would produce it — but the explanation has to be concrete.

The requirements

Fast and deterministic. One function: `(semantic_model, contract, dialect) -> sql_string`. Under ~10 milliseconds per contract, same output every run — measure it and report your numbers. No network calls, no LLM calls, nothing non-deterministic anywhere in the path.

More than one dialect. The emitted SQL must run on DuckDB, and your solution must also emit at least one other dialect (Postgres or Snowflake — your pick) without rewriting the part that understands contracts. We want to see where you draw that line, not exhaustive dialect coverage.

Correct join semantics. Attaching a dimension must not drop facts (a sale whose store is missing from `dim_store` is still revenue), but a filter on a dimension's field genuinely restricts. Say in your README which join types you chose and why.

Honest failure. A contract naming an unknown metric or dimension, or asking for something your solution doesn't support, must produce a clear, named error — never silently-wrong SQL. We would much rather see a good refusal than a bad answer.

Tests. Enough that you would trust a refactor. The expected results above are your first fixtures.

Any language. Any open-source libraries, or none — what you reach for is itself a decision we will ask about, and if a library does heavy lifting for you, expect questions on what it is doing underneath.

Deliverables

- The code, runnable with one command against DuckDB.
- Tests, runnable with one command.
- A README with your reasoning — this carries as much weight as the code:
 - the shape of your solution and why you chose it,
 - how you would add a new contract feature — say, a `having` clause or a month-by-month trend — without rewriting what exists,
 - what breaks first if the contract vocabulary grows 10×, and what you would change then,
 - your measured generation times,
 - anything you cut, and what you would do next.

How we evaluate

Roughly in this order:

- **Structure** — could a second engineer read your solution, extend it, and trust it?
- **Semantic correctness** — the totals rows, the join types, the conditional aggregation. These are where analytics systems quietly lie, and noticing that is the job.
- **The written reasoning** — the README tells us more than the code volume does.
- **Honest failure handling** — what happens off the happy path.
- **Performance** — mostly as evidence the approach is right; we are not timing you to the microsecond.

In the follow-up conversation we will extend the problem together — new question shapes, a second fact table, bigger models — and talk through how your solution absorbs each one.